

LAB3. Upravljanje sustavom korištenjem višedretvenosti

Složeniji sustavi se jednostavnije upravljaju ako se koriste dretve za pojedini element upravljanja. Međutim, takve sustave treba oprezno pripremiti da bi sve dretve ispravno radile (stigle sa svojim poslovima do krajnjeg trenutka dovršetka, tj. pojave idućeg takvog događaja nakon jedne periode).

Zadatak ove vježbe je da se za svaki ulaz stvori zasebna upravljačka dretva koja će njime upravljati. Njen idejni kod prikazan je u nastavku.

```
dretva upravljač za ulaz x
{
    t = t0 + 5 ms //trenutak prve pojave + malo

    ponavljaj {
        spavaj_do (t) //čekaj početak iduće periode, koristiti sleep!

        ako je ulaz promijenio stanje {
            obradi zadatak x //radno čekanje: "simuliraj x ms" prema uputama ispod
        }

        t += T
    }
}
```

Obzirom da se ovdje radi o višedretvenoj obradi (za razliku od prijašnjih vježbi), ovdje protok vremena u upravljačkim dretvama ne simulirati na isti način kao u prethodnim vježbama (sa sleep), već koristiti radno čekanje - dretve troše stvarno procesorsko vrijeme. Na početku rada simulatora ispitati koliko je potrebno puta odvrtni jednu petlju za npr. 10 ms vremena. Taj broj kasnije koristiti u simulaciji. Skica ideje je prikazana u nastavku.

```
troši 10 ms {
    //broj - globalna varijabla
    za i = 1 do broj {
        ništa ili nešto, ali da kompajler to ne makne,
        npr. u C-u: asm volatile("" ::: "memory");
    }
}

odredi-broj-iteracija-za-10-ms {
    broj = 1000000 //ili slično
    vrti = 1
    dok je vrti == 1 {
        t0 = dohvati_vrijeme()
        troši 10 ms //poziv gornje funkcije
        t1 = dohvati_vrijeme()
        ako je t1 - t0 >= 10 ms
            vrti = 0 //traje dovoljno dugo
        inače
            broj *=10 //traje prekratko
    }
    broj = broj * 10 / (t1-t0)
}

simuliraj x ms {
    za i = 1 do x/10 {
        troši 10 ms
    }
}
```

Simulacija trošenja X jedinica vremena petljom:

```
t0 = dohvati_vrijeme()
dok je dohvati_vrijeme() < t0 + X radi
    ništa
```

nije dobro - dretve bi tako paralelno čekale protok vremena (kao da sve paralelno rade).

Korištenjem "normalnih" dretvi moglo bi se dogoditi da se dretve različitih zadataka paralelno izvode (zaista paralelno na višeprocesorskim računalima ili kvazi-paralelno korištenjem podjele vremena). U SRSV-ima se najčešće dretvama (s takvim periodičkim ograničenjima) dodjeljuju prioriteti te se najprije izvode dretve najvećeg prioriteta (možda i nekoliko njih paralelno na višeprocesorskim sustavima).

U simulaciji koristi raspoređivanje SCHED_FIFO. (Preporuka je da se vježba najprije napravi i ispita bez promjene načina raspoređivanja i tek onda nadogradi.)

Za pokretanje programa koji koristi SCHED_FIFO trebaju administratorske ovlasti. Također, oprezno pokretati te programe jer oni mogu zauzeti svo procesorsko vrijeme (npr. u početku postaviti kratko vrijeme simulacije, a kasnije ga povećavati).

Prioritete dodijeliti prema mjeri ponavljanja (dretvama s kraćim periodama veći prioritet). Potrebna sučelja za postavljanje načina raspoređivanja te parametre dretvi su prikazani u skripti u 5. poglavlju.

Simulacija ulaza treba biti jednaka kao u zadatku LAB1, ali može se staviti značajno manje zadataka (4-10). Dretve koje simuliraju ulaze također raspoređivati prema SCHED_FIFO ali s najvećim prioritetom (sve s istim), većim od svih upravljačkih dretvi (one ne smiju raditi radno čekanje već imati "sleep" unutar petlje!).

Na kraju simulacije ispisati statistiku.

Na višeprocesorskim sustavima (i na višejezgrenim procesorima) će očito biti "lakše" zadovoljiti vremenska ograničenja jer se obrade mogu obavljati paralelno.